

Junghyun Min

jm3743@georgetown.edu | [Google Scholar](#) | [Personal website](#) | [GitHub](#) | Washington, DC | (443) 414-4405

SKILLS & SERVICE

Software Engineering: Unix, databases, Flask, FastAPI, asynchrony, Google Cloud, Docker, Portainer, Git, Jenkins.
Natural Language Processing: PyTorch, TensorFlow, transformers, TorchServe, scikit-learn, ML baselines
Large Language Models: OpenAI, LangChain, RAG, quantization, distributed training, vLLM, ollama, autoresearch
Languages: Python, Java, C++, JavaScript, R, Wolfram; English/Korean (Native), German/Mandarin (Working)
Service: Reviewer for *ACL, LREC, MLE, NeurIPS Workshops, ACL Workshops; NACLO Site Host

EDUCATION

Georgetown University Washington, DC
Ph.D. Computer Science and Linguistics. Advisor: Ethan Wilcox. Exp. 2029

Johns Hopkins University Baltimore, MD
M.A. Cognitive Science. Advisor: Tal Linzen. Dec 2020
B.S. Physics, B.A. Mathematics. General Honors, early graduation. Dec 2017

WORK EXPERIENCE

Nokia Bell Labs Murray Hill, NJ
Machine Learning and AI Intern at AI Research Lab Jun 2026 – Aug 2026

- Prototyped agentic autonomous ML research for open-ended document retrieval, accelerated development by 10x
- Selected as a finalist (12 out of 200 interns) to present to Bell Labs leadership, received Outstanding Innovation Award

University of Toronto Toronto, ON
Computer Science Research Intern May 2025 – Aug 2025

- Pre-trained LMs from scratch and via transfer; compiled 7-task Cantonese NLU benchmark and established baselines
- Performed large-scale verification of a novel training objective across architecture, language, and resource settings

NCSoft Seongnam, Korea
Machine Learning Engineer (Natural Language Processing) at Language AI Lab Jan 2021 – Apr 2024

- Implemented and trained tokenizers for the Language Model Task Force, precursor to the 1.3-13B [VARCO LLMs](#)
- Deployed efficient, scalable Stanza-like cloud REST API, parsing 10k requests per sec on 4GB VRAM at 96% acc.
- Served machine learning models for information retrieval via Docker and TorchServe, processing 30+ sent per sec.
- Designed algorithms in deep learning models to improve information extraction performance (+30%p) and stability
- Built and optimized agentic pipelines to automatically parse current news to extract and rank 1,000 events per day

SELECTED TECHNICAL PROJECTS

Lead, Autonomous research for open-ended ML problems

- Implemented scripted harness to extend agentic autonomous research frameworks to open-ended retrieval problems
- Matched stability and performance of commercial agents (Claude, Cursor) using locally served open-source LLMs

Lead engineer, Agentic document automation platform [genDOC](#)

- Built RAG-based agentic document automation software with LangChain and FastAPI on OpenAI API using Python
- Implemented orchestration framework, routing pipeline, agent-wise output verification logic and self-correction loops
- Optimized RAG pipeline to achieve 90+% end-to-end pass rate (pass@5) and end-to-end latency under 60 seconds

Co-lead, [Large language models and legal interpretation](#)

- Implemented LLM legal interp. judgment extraction tool with vLLM. [Oral presentation](#), NLLPW at EMNLP 2025
- Showed even large (70B) and commercial LLMs are sensitive to surface form, lack alignment to human judgment

Lead, Multimodal text-prosody model for syntactic information content estimation

- Developed and fine-tuned 3-modal transformer architecture for syntax prediction with multimodal text and audio input
- Measured syntax information content in text (-74%), duration (-3%), pause (-1.7%) in naturalistic and planned speech

SELECTED PUBLICATIONS

- Junghyun Min, et al. RepL4NLP at NAACL 2025. [Punctuation Restoration Improves Structure Understanding without Supervision](#). [Tech blog \(Korean\)](#). PyTorch implementation on Docker.
- Abhishek Purushothama and Junghyun Min et al. FAccT 2026. [Prompting from the bench: Large-scale pretraining is not sufficient to prepare LLMs for ordinary meaning analysis](#). vLLM and OpenAI API.
- Junghyun Min, et al. EMNLP 2026. [Using Prosody to Predict Syntactic Structure](#). PyTorch implementation.